Canonical Subproblems for Robot Inverse Kinematics

Alexander J. Elias and John T. Wen, *Fellow, IEEE*

arXiv:2211.05737v2 [cs.RO] 19 Jun 2023

***Abstract*—The inverse kinematics (IK) problem for many common robot manipulators may be decomposed into canonical subproblems which are solved by finding the angles on circles where they intersect with other geometric objects. We present new algebraic solutions and geometric interpretations for six subproblems using a linear algebra approach, and we demonstrate significant computational performance improvements over existing IK methods. We show that IK for any 6-dof all revolute (6R) robot with three intersecting or parallel joint axes may be solved in closed form using subproblem decomposition. For any other 6R robot, subproblem decomposition reduces finding all IK solutions to a search over one or two joint angles. The first three subproblems, called the Paden–Kahan subproblems, are Subproblem 1: Circle and Point, Subproblem 2: Two Circles, and Subproblem 3: Circle and Sphere. The other three subproblems, which have not been extensively covered in the literature, are Subproblem 4: Circle and Plane, Subproblem 5: Three Circles, and Subproblem 6: Four Circles. Our approach also finds the least-squares solutions for Subproblems 1–4 when the exact solution does not exist.**

***Index Terms*—Computational Geometry, Industrial Robots, Kinematics, Paden–Kahan Subproblems.**

I. Introduction

ROBOTICS applications have always demanded a fast and reliable inverse kinematics (IK) algorithm which can ideally return all IK solutions for a given pose. Gradient-based solvers, such as the popular solver TRAC-IK [1], use the robot Jacobian and can run sufficiently quickly for real-time control. However, they only return solutions closest to the initial guess, suffer from performance issues close to robot singularities, may not run fast enough for certain real-time path planning applications, and may not always return solutions within a given time. In contrast, analytical IK solvers can return all inverse kinematics solutions and can run much faster than gradient-based solvers. The most widely used analytical IK solver, IKFast [2], has been used in large projects such as MoveIt [3]. However, IKFast depends on symbolic manipulation techniques, meaning code generation (and even compilation) can take a considerable amount of time, and there is no guarantee of solutions for certain robot kinematics. Furthermore, the generated code is extremely lengthy, meaning modifying, optimizing, or debugging by hand is impractical.

In this paper, we revisit a geometric approach to inverse kinematics capable of finding all IK solutions called subproblem decomposition. This method solves for a small number of joint angles at a time by rewriting the problem as a series of a few canonical subproblems with known solutions. This results in extremely efficient algorithms which can be implemented in few lines of code and can be easily modified, optimized, and debugged. Furthermore, the same code can apply to any robot in the same kinematic family, even if the exact link lengths or joint offsets are different.

TABLE I
Subproblem Formulations

Subproblem	Equation
1: Circle and Point	$\min \lVert \boldsymbol{R}(k,\theta)p_1 - p_2 \rVert$
2: Two Circles	$\min \lVert \boldsymbol{R}(k_1,\theta_1)p_1 - \boldsymbol{R}(k_2,\theta_2)p_2 \rVert$
3: Circle and Sphere	$\min \lvert \lVert \boldsymbol{R}(k,\theta)p_1 - p_2 \rVert - d \rvert$
4: Circle and Plane	$\min \left\lvert h^T \boldsymbol{R}(k,\theta)p - d \right\rvert$
5: Three Circles	$p_0 + \boldsymbol{R}(k_1,\theta_1)p_1 = \boldsymbol{R}(k_2,\theta_2)(p_2 + \boldsymbol{R}(k_3,\theta_3)p_3)$
6: Four Circles	$\begin{cases} h_1^T \boldsymbol{R}(k_1,\theta_1)p_1 + h_2^T \boldsymbol{R}(k_2,\theta_2)p_2 = d_1 \\ h_3^T \boldsymbol{R}(k_3,\theta_1)p_3 + h_4^T \boldsymbol{R}(k_4,\theta_2)p_4 = d_2 \end{cases}$

We identify three new canonical subproblems in addition to the three original ones and provide alternate solution methods based on geometry and linear algebra. These subproblem solutions do not rely on any computer-assisted algebra or the tangent half angle substitution $x = \tan(\theta/2)$. Instead, the subproblems are reduced to common linear algebra problems like finding least-squares and minimum norm solutions to linear equations. Subproblems 5 and 6 are reduced to solving the intersection of two ellipses, which may be solved by finding roots of quartic polynomials.

Using these subproblem solutions, we propose a closed-form method for inverse kinematics for any 6-dof all-revolute (6R) robot with three consecutive parallel or intersecting axes (commonly referred to as the Pieper criterion [4]), only requiring addition, subtraction, multiplication, division, square roots, ATAN2, and sometimes cube roots. The subproblem decomposition method is robust to internal singularities and in some cases (solutions which only use Subproblems 1–4) robust to boundary singularities.

We also describe a search-based method for solving more general robots. For 6R robots with at least one pair of parallel or intersecting axes, the problem can be solved using a 1D search over one joint angle, and for the most general 6R case, IK can be solved using a 2D search over two joint angles. Due to the curse of dimensionality, searching over just one or two joint angles is much more efficient than searching over all six joints as in Jacobian-based methods.

The subproblems considered in this paper are summarized

This work is supported in part by NASA Goddard SEAS contract NNG15CR66C, Subcontract ATSJV-D-38008-019, Task 0049. *(Corresponding author: Alexander J. Elias.)*

The authors are with the Department of Electrical, Computer, and Systems Engineering, Rensselaer Polytechnic Institute, Troy, NY 12180 USA (e-mail: eliasa3@rpi.edu; wenj@rpi.edu).

in Table I, where θ is an unknown angle, p is an $\mathbb{R}^3$ vector, h is a unit $\mathbb{R}^3$ vector, and d is a scalar. (d must be nonnegative in Subproblem 3.) We pose Subproblems 1–4 as least-squares problems, while Subproblems 5 and 6 are posed as exact equations. We also consider generalizations of Subproblem 2 where the circle axes do not intersect. Note that $\boldsymbol{R}(k,\theta)$ denotes the rotation matrix about a unit vector $k \in \mathbb{R}^3$ over an angle θ. For some fixed p, k and parameter θ, $\boldsymbol{R}(k,\theta)p$ sweeps out the circle on the edge of a right circular cone with axis k and generator p. As long as there is no continuum of solutions, Subproblem 1 has one exact solution or one least-squares solution, Subproblem 2 has up to two exact solutions or up to two least-squares solutions, Subproblems 3 and 4 have up to two exact solutions or one least-squares solutions, and Subproblems 5 and 6 have up to four solutions.

We provide an overview of the history of inverse kinematics and the subproblem decomposition method in section II, and then explain our approach in section III. In section IV we provide efficient and numerically robust solutions to the six canonical subproblems. We explain conditions under which the subproblems may be decomposed further, and explain conditions where the solutions follow a continuum of values, which in the application of robot IK correspond to internal singularities. In section V, we provide a number of IK solutions which utilize the subproblem solutions, and in section VI we demonstrate significant performance speedups compared to alternatives like IKFast. We conclude and discuss ideas for improving the algorithms even further in section VII.

The subproblem and IK solution programs along with timing tests are coded in MATLAB and Rust, and they are available on a publicly accessible repository[1].

II. Related Work

This work is a continuation of a long line of work in analytical and semi-analytical (i.e., reduced-dimension search) solutions to the inverse kinematics problem. The seminal thesis by Pieper in 1969 [4] presented analytical solutions for many special robot cases, and for the general 6R robot case, he showed the characteristic polynomial is at most degree 524,288. Since then, work on the IK problem by other authors has tended to focus on two areas: The general 6R robot IK problem and simplified kinematics IK problems.

To solve the general 6R robot IK problem, most authors find a 16th order polynomial whose real roots correspond to the (up to) 16 solutions for such an arm. Although the general 6-dof IK problem has been solved, solutions often require computer-assisted algebra or the tangent half angle substitution, so there is still a need for more robust, faster, and more easily understood algorithms. Much of this work was published in the early 1990s. Raghavan and Roth [5], [6] were one of the first to devise a procedure to construct a 16th order polynomial in the tangent half angle of one of the joints. Many others quickly followed with simpler and more efficient procedures. [7] found a 16th order polynomial in a different manner, [8] reduced general 6R IK to an eigenvalue problem or generalized eigenvalue problem, and [9] reduced the problem to a 12×12 quadratic eigenvalue problem. More recently in 2007, [10] solved the general 6R IK problem using dual quaternions and 7D projective geometry. They reduced the IK problem to the intersection of 8 hyperplanes and a six-dimensional quadric, which is solved by finding 48-degree polynomial and factoring out a 16th order polynomial. A 2D search-based method is discussed in [11] (they call it the bivariate-equation approach) where two joint angles are searched over, and the tangent half angle substitution is not used.

For certain robot geometries, there are fewer than 16 solutions, and the order of the characteristic polynomial decreases [12]. In some cases, the solution may be simplified down to solving quartic or even quadratic polynomials. While using the general 6R methods for simplified robot cases is viable, the solutions tend to be much easier when using methods specific to those cases. Pieper [4] focused much of his work on the cases of three intersecting axes because the rotation and position parts of the IK problem become decoupled. The three parallel axes case is similar, as the three axes still intersect but at a point at infinity.

Paden [13] showed that for some of these special 6R robots, the inverse kinematics problem may be decomposed into a series of smaller canonical subproblems with closed-form solutions. The influential robotics textbook [14] called these subproblems the Paden–Kahan Subproblems 1, 2, and 3. However, Subproblems 1–3 alone cannot be used to solve all 6R robots with three parallel or intersecting axes. We identify Subproblems 5 and 6 as the canonical problems solved in the case of three intersecting or parallel axes, respectively. Subproblem 5 has been identified by many names including the position problem [11], the third order subproblem [15], the three-joint subproblem [16], and the wrist center positioning problem [17]. We also identify Subproblem 4 as a canonical subproblem, as it is used when there are two parallel axes in a robot or in solving a spherical wrist. It is important to identify these subproblems as they may be used to solve geometric problems other than the 6R robot IK problem, including the IK problem for 6-dof robots with prismatic joints or the IK problem for 7-dof robots.

Various authors have contributed solutions to the original three Paden-Kahan Subproblems as well as Subproblems 4–6, either by explicitly identifying them as subproblem solutions or by addressing them while solving IK problems. (Authors tend to solve the exact versions rather than the least-squares versions of these subproblems.) In [15], [16], [18]–[32], subproblems are solved using screw theory. In [29], subproblem solutions are provided with quaternions, and in [29], [33]–[35] dual quaternions are used. In [19], [21], [22], [28], [32], a new subproblem of two consecutive rotations with an offset is posed, which is a generalization Subproblem 2. In [23], [26], [32], [35], the solution for two consecutive parallel rotations is provided, which is a special case of this generalization. Subproblem 4 has been solved before as part of the solution to the inverse kinematics of a spherical wrist, such as in [36]. Several authors posed new subproblems with three consecutive rotations with constraints on which axes are parallel or intersecting, which are special cases of Subproblem

[1] https://github.com/rpiRobotics/linear-subproblem-solutions

5 [18], [20], [23], [24], [26], [28], [32], [34]. In [11], [15]–[17], the fully general case of three consecutive rotations is solved. In [37], a specific instance of Subproblem 5 was solved for IK of a 7R arm, and the authors noticed that the problem reduces to the intersection of two ellipses. The general 6-dof methods presented in [10] simplify when a robot has three intersecting or parallel axes, meaning this method can solve Subproblems 5 and 6, as elaborated on in [38]. In solving Subproblem 5, the simplifying cases of two parallel or intersecting axes means authors also solve Subproblems 2 and 4. In [11], the author reduces Subproblem 5 to Subproblem 6, and so inadvertently solves Subproblem 6 as well.

To demonstrate the computational performance and range of robot kinematics that the subproblem decomposition method offers, we will compare performance to IKFast and to a MATLAB implementation of Pieper's method. IKFast [2] is a widely used analytical inverse kinematics solver which uses a Python script to generate inverse kinematics C++ code. Although it is part of the OpenRAVE project, IKFast is also used in large projects like MoveIt [3] and Tesseract [39], as well as smaller standalone projects. The original implementation could find IK solutions for robots with three intersecting axes by decomposing the kinematics into a position and rotation part and solving them separately using symbolic manipulation. More recent updates to the program code also include implementations of the general 6-dof IK algorithms presented in [7]–[9]. MATLAB's Robotics System Toolbox [40] closed-form inverse kinematics solver, `analyticalInverseKinematics`, is based on the Pieper method [4] and is designed to work with any robot with a spherical wrist. The solver generates a MATLAB function which finds all inverse kinematics solutions, which can then be converted to a C++ MEX function with MATLAB Coder.

III. Approach

We solve Subproblems 2 and 3 by reducing them to Subproblem 4, and we solve Subproblem 5 by manipulating the Subproblem 4 solution. We solve the remaining Subproblems 1, 4, and 6 by writing them as a linear equations in $x = \begin{bmatrix} \mathrm{s}_\theta & \mathrm{c}_\theta \end{bmatrix}^T$, where $\mathrm{s}_\theta = \sin\theta$ and $\mathrm{c}_\theta = \cos\theta$, with nonlinear constraint $\sin^2(\theta) + \cos^2(\theta) = \|x\|^2 = 1$. We first solve the unconstrained linear problem with the tools of linear algebra, and then find the subset of solutions following the constraint. A similar approach was used in [41] to find intersections among spheres. To convert a subproblem to a linear problem, we start with the Euler-Rodrigues formula for rotations [42], [43]:

$$\boldsymbol{R}(k,\theta) = kk^T + \mathrm{s}_\theta k^\times - \mathrm{c}_\theta k^{\times 2}, \tag{1}$$

where $k^\times$ denotes the matrix representation of the cross product. For $k = [k_1, k_2, k_3]^T$,

$$k^\times := \begin{bmatrix} 0 & -k_3 & k_2 \\ k_3 & 0 & -k_1 \\ -k_2 & k_1 & 0 \end{bmatrix}, \tag{2}$$

and $k^{\times 2} = k^\times k^\times = kk^T - I$. We may rewrite (1) as

$$\boldsymbol{R}(k,\theta)p = p_k + A_{k,p}x, \tag{3}$$

where

$$p_k = kk^T p \quad \text{and} \quad A_{k,p} = \begin{bmatrix} k^\times p & -k^{\times 2} p \end{bmatrix}. \tag{4}$$

Geometrically, this conversion replaces any circle with the plane in which the circle lies. The vector p_k is along the circle axis pointing to the circle center, and the columns of $A_{k,p}$ form an orthogonal basis for the plane and have norm equal to the circle radius.

In Subproblems 1–4, if an exact solution does not exist, then we instead find the solution which minimizes Euclidean distance. Conveniently, this least-squares solution requires almost no overhead to compute. The minimizing solutions between two smooth objects occurs on a line normal to both objects. As pointed out in [44], for a circle the common normal passes through the circle axis.

Given a solution for x, the corresponding angle is found using $\theta = \mathrm{ATAN2}(x_1, x_2)$. ATAN2 is the only inverse trigonemtric function used as it is extremely robust compared to $\sin^{-1}()$ and $\cos^{-1}()$ [2], as well as compared to solving for $\tan(\theta/2)$ [9], [11]. We can achieve some computational speed-up by recognizing $\mathrm{ATAN2}(\mathrm{s}_\theta, \mathrm{c}_\theta) = \mathrm{ATAN2}(\alpha \mathrm{s}_\theta, \alpha \mathrm{c}_\theta)$ where $\alpha > 0$. This means we only have to solve for αx.

In the IK solutions and when we reduce subproblems to other subproblems, we solve the equation $\boldsymbol{R}(k,\theta)p_1 = p_2$ (where p_1 and p_2 may be functions of other unknowns) by solving the necessary condition $\|p_1\| = \|p_2\|$ or $k^T p_1 = k^T p_2$. In general, these conditions are not sufficient, and so solutions must be checked if they are extraneous.

IV. Subproblem Solutions

Subproblem 1: Circle and Point

Given vectors p_1, p_2 and a unit vector k, find θ to minimize $\|\boldsymbol{R}(k,\theta)p_1 - p_2\|$.

Assume k and p_1 are not collinear, and assume k and p_2 are not collinear. Otherwise, θ is arbitrary. Using (3), we have

$$\|\boldsymbol{R}(k,\theta)p_1 - p_2\| = \|A_{k,p_1}x - b\|, \tag{5}$$

where $b = p_2 - p_{1k} = p_2 - kk^T p_1$. By assumption, A_{k,p_1} has full column rank. An exact solution to $A_{k,p_1}x = b$ such that $\|x\| = 1$ exists when p_2 lies on the circle, meaning $\|p_1\| = \|p_2\|$ and $k^T p_1 = k^T p_2$. In this case, x is the unconstrained least-squares solution

$$x_{LS} = \min_x \|A_{k,p_1}x - b\| = A^+_{k,p_1} b = A^+_{k,p_1} p_2, \tag{6}$$

where $A^+ = (A^T A)^{-1} A^T$ is the pseudo-inverse (left inverse) of A. One can show $A^+_{k,p_1} = A^T_{k,p_1} / \|k^\times p_1\|^2$.

In general, an exact solution is not possible, and we need to solve the constrained least-squares problem $\min_x \|A_{k,p_1}x - b\|$ subject to $\|x\| = 1$. Since the common normal between the circle and p_2 must pass through the circle's axis, the minimizing solution occurs on the plane of symmetry containing the circle's axis and p_2. As shown in Fig. 1, x_{LS} is the coordinates in the coordinate system $(-k^{\times 2}p_1, k^\times p_1)$ of p_2 projected onto the plane containing the circle. This vector

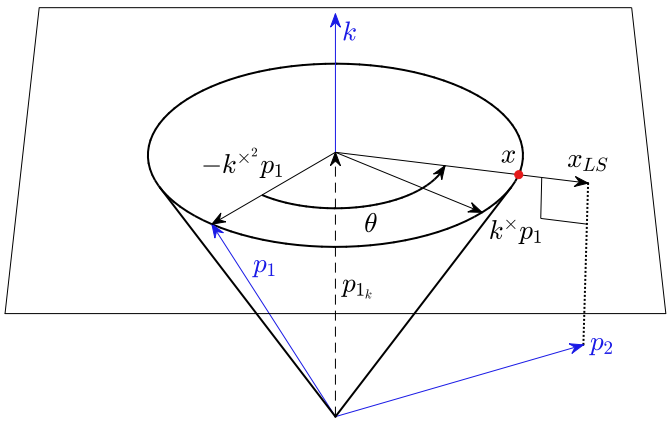


Fig. 1. Subproblem 1 finds the angle of the closest point on the circle $\boldsymbol{R}(k,\theta)p_1$ to the point p_2. This is the angle of x_{LS} in the coordinate frame in the circle plane formed by $(-k^{\times^2}p_1, k^{\times}p_1)$, where x_{LS} is the projection of p_2 to the plane containing the circle.

is in the plane of symmetry, and normalizing it gives the constrained least-squares solution $x = x_{LS}/\|x_{LS}\|$. Skipping all positive scalar division (which keeps the value of ATAN2 unaffected) yields the general solution

$$\theta = \text{ATAN2}((k^{\times}p_1)^T p_2, -(k^{\times^2}p_1)^T p_2). \tag{7}$$

Note that all other p_2 vectors which lie in the same half plane containing k result in the same least-squares solution.

SUBPROBLEM 2: TWO CIRCLES

Given vectors p_1, p_2 and unit vectors k_1, k_2, find (θ_1, θ_2) to minimize $\|\boldsymbol{R}(k_1,\theta_1)p_1 - \boldsymbol{R}(k_2,\theta_2)p_2\|$.

Assume the following pairs of vectors are not collinear: (k_1, p_1), (k_2, p_2), (k_1, k_2). Otherwise, the problem reduces to Subproblem 1 and θ_1, θ_2, or $\theta_1 \pm \theta_2$ are arbitrary, respectively.

An exact solution $\boldsymbol{R}(k_1,\theta_1)p_1 = \boldsymbol{R}(k_2,\theta_2)p_2$ is possible only if $\|p_1\| = \|p_2\|$, as this means the two circles lie on the same sphere. We see the minimizing (θ_1, θ_2) is independent of $\|p_1\|$ and $\|p_2\|$ if we rewrite the subproblem as

$$\begin{aligned}\|\boldsymbol{R}(k_1,\theta_1)p_1 - \boldsymbol{R}(k_2,\theta_2)p_2\|^2 \\ = \|p_1\|^2 + \|p_2\|^2 - 2\|p_1\|\|p_2\|\cos(\psi),\end{aligned} \tag{8}$$

where ψ is the angle between the rotated vectors. This means we can find the least-squares solution(s) by normalizing p_1 and p_2 before proceeding. Once we assume p_1 and p_2 are the same length, an exact solution exists if and only if

$$|\theta_{k_1,p_1} - \theta_{k_2,p_2}| \le \theta_{k_1,k_2} \le \theta_{k_1,p_1} + \theta_{k_2,p_2}, \tag{9}$$

where θ_{v_1,v_2} is the angle between vectors v_1 and v_2. This inequality can be derived by considering the intersections of the circles with the plane of symmetry spanned by k_1 and k_2. A solution exists if and only if the intersections are interleaved, that is, the intersections alternate between circles 1 and 2 as we rotate along the plane of symmetry. If p_1 is arbitrary, then θ_{k_1,p_1} can be any angle from 0 to π, and a solution always exists if and only if $k_1^T k_2 = 0$ and $k_2^T p_2 = 0$.

Projecting the Subproblem 2 equation onto k_2 or k_1 gives equations only in terms of θ_1 or θ_2 which can be solved using Subproblem 4:

$$k_2^T \boldsymbol{R}(k_1,\theta_1)p_1 = k_2^T p_2, \tag{10a}$$
$$k_1^T \boldsymbol{R}(k_2,\theta_2)p_2 = k_1^T p_1. \tag{10b}$$

If there are two solutions for θ_1 and θ_2, ensure the solutions match by switching the order of solutions for θ_2. Sometimes θ_1 and θ_2 may have different numbers of solutions due to numerical issues; in this case, duplicate the solution for the angle with only one solution. Subproblem 4 finds the intersections between each circle and the plane in which the other circle lies, as shown in Fig. 2(a). Since the two circles lie on the same sphere. and there are no extraneous solutions: each intersection between a circle and the other plane must also be an intersection with the other circle.

If the two circles do not intersect, as in Fig. 2(b), the least-squares solution is also given by Subproblem 4, and this case can be detected by checking if Subproblem 4 returns a least-squares result. The minimizing solutions occur when line passing through $\boldsymbol{R}(k_1,\theta_1)p_1$ and $\boldsymbol{R}(k_2,\theta_2)p_2$ passes through both circle axes, meaning the solutions must occur on the plane of symmetry containing k_1 and k_2. (We have removed the case of the line being out of plane but passing through the origin by assuming $\|p_1\| = \|p_2\|$.) The least-squares solution for Subproblem 4 lies on this same plane.

Remark 1 (Alternate Decomposition). Alternatively, solve Subproblem 4 for one angle, then use Subproblem 1 to find the remaining angle solutions.

Remark 2 (Linear Solution). This subproblem may be solved by rewriting the problem as a linear equation $Ax = b$, where

$$A = \begin{bmatrix} A_{k_1,p_1} & A_{k_2,p_2} \end{bmatrix}, \text{ and } b = p_{2_k} - p_{1_k}. \tag{11}$$

The complete solution $x = x_{min} + x_N$ is the intersection between the two planes, where x_{min} is the minimum-norm solution and x_N is an arbitrary vector in the one-dimensional null space of A. Simplifying to find efficient expressions for x such that $\|x_i\| = 1$ results in the solution above.

Remark 3 (Skew Axes). It is straightforward to extend this subproblem to the case where the circle axes are skew, solving

$$p_0 + \boldsymbol{R}(k_1,\theta_1)p_1 = \boldsymbol{R}(k_2,\theta_2)p_2. \tag{12}$$

The solution is nearly identical: Project onto k_1 or k_2 and solve Subproblem 4 and Subproblem 1 without normalizing p_1 and p_2. Since the axes are skew, there is only up to one solution, and we need to check for extraneous solutions by checking if both subproblems returned an exact solution. This problem can be solved in a similar way using Subproblem 3 rather than Subproblem 4 by taking the norm of both sides of the equation. Solving the least-squares version of this problem is possible but complicated as there are up to eight common normals between two circles [45].

When the circle axes are skew, the two circles lie on different spheres, and the problem is generally unsolvable: with randomly chosen parameters, there is almost surely no

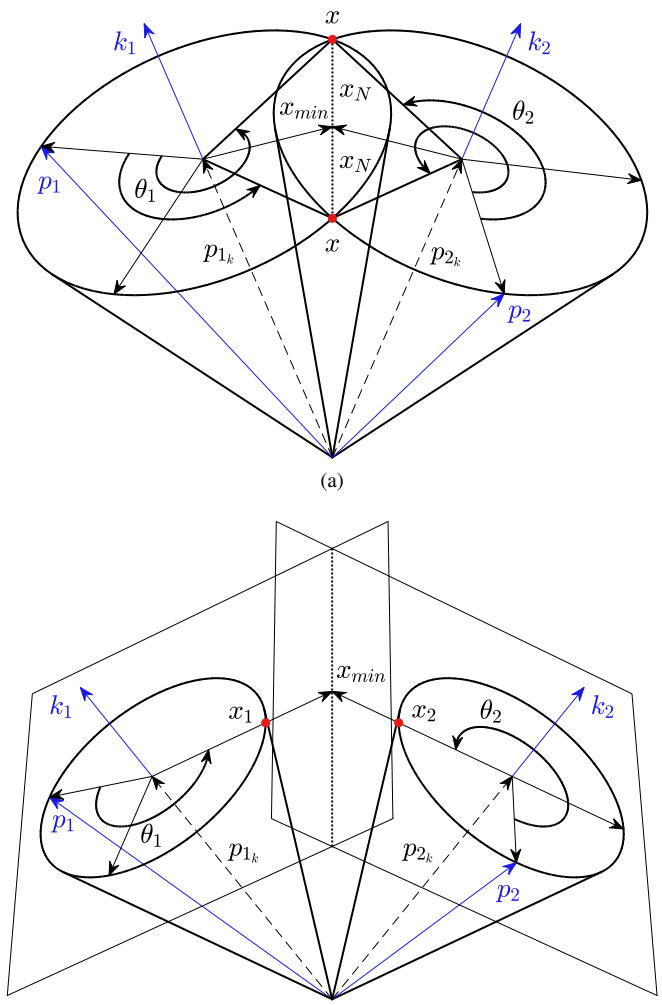


Fig. 2. Subproblem 2 finds the angles of the closest points on the two circles $\boldsymbol{R}(k_1, \theta_1)p_1$ and $\boldsymbol{R}(k_2, \theta_2)p_2$. (a) Two intersecting circles with two intersection points. The solutions are found along the intersection of the planes containing the circles. (b) Two non-intersecting circles. The minimizing solution is the angle of the minimum-norm point on the intersection of the two planes containing the circles.

solution for (θ_1, θ_2). It is therefore inadvisable to use this as part of the solution to a larger geometry problem.

Remark 4 (Intersecting Axes). If p_0 can be written as a linear combination of k_1 and k_2, then the circle axes still intersect, and the problem can be solved with Subproblem 2 by moving the respective components of p_0 to p_1 and p_2.

Remark 5 (Parallel Axes). If the circle axes are parallel ($k_1 = k_2 = k$), we require $k^T(p_0 + p_1 - p_2) = 0$ to guarantee exact solutions. Up to two solutions of θ_1 may be solved using Subproblem 3, and the corresponding solutions of θ_2 may be found using Subproblem 1. We can also find up to two least-squares solutions if $k^T(p_0 + p_1 - p_2) \neq 0$ by replacing p_i with $-k^{\times 2}p_i$ for $i = 0, 1, 2$.

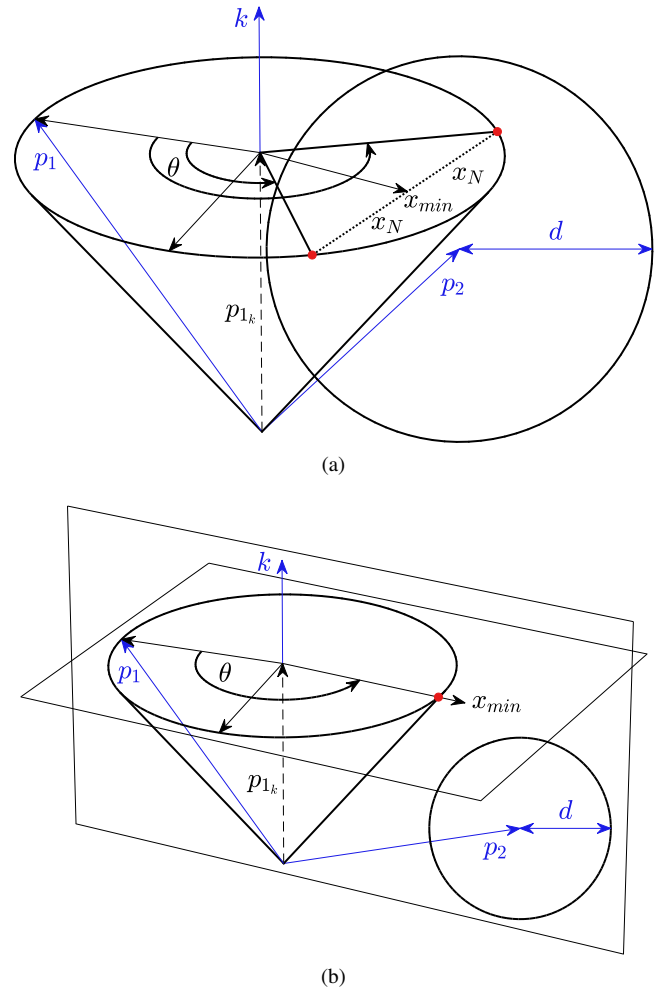


Fig. 3. Subproblem 3 finds the angles of the closest points on the circle $\boldsymbol{R}(k, \theta)p_1$ to the sphere of radius d centered at p_2. (a) Intersecting circle and sphere. (b) Minimum distance between a non-intersecting circle and sphere.

SUBPROBLEM 3: CIRCLE AND SPHERE

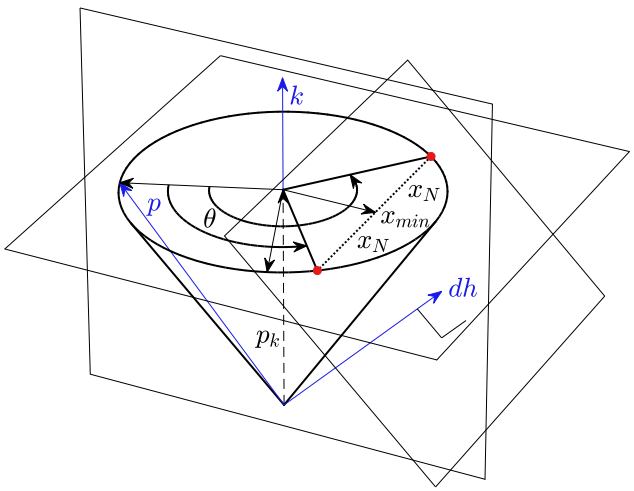
Given vectors p_1, p_2, a unit vector k, and a nonnegative scalar d, find θ to minimize $|\|\boldsymbol{R}(k, \theta)p_1 - p_2\| - d|$.

Assume k and p_1 are not collinear, and assume k and p_2 are not collinear; otherwise, the problem is independent of θ.

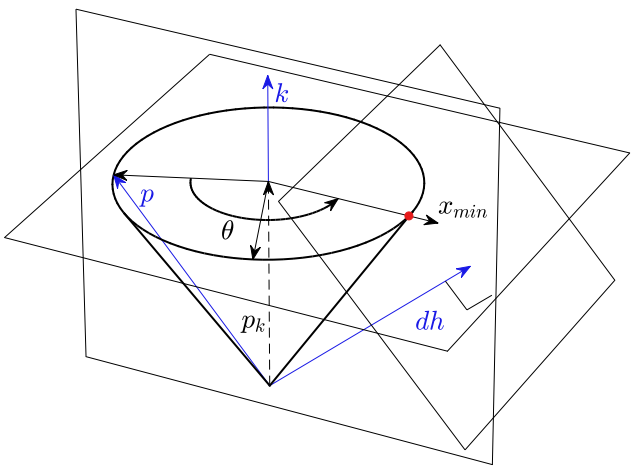
Write the problem equivalently as minimizing

$$\left|\frac{1}{2}\|\boldsymbol{R}(k, \theta)p_1 - p_2\|^2 - \frac{d^2}{2}\right| = \left|p_2^T \boldsymbol{R}(k, \theta)p_1 - \frac{1}{2}(\|p_1\|^2 + \|p_2\|^2 - d^2)\right|, \quad (13)$$

which can be solved using Subproblem 4. If the circle and sphere intersect, as in Fig. 3(a), then this conversion, which is basically the law of cosines, replaces the sphere with a plane which intersects the circle at the same points. If the circle and sphere do not intersect, as in Fig. 3(b), the minimizing angle on the circle to the plane is also the minimizing angle to the sphere, which occurs on the line passing through the circle axis and the center of the sphere. Interestingly, if $d = 0$ then this subproblem solves Subproblem 1.



(a)



(b)

Fig. 4. Subproblem 4 finds the angles of the closest points on the circle $\boldsymbol{R}(k,\theta)p$ to the plane with normal vector h and distance d from the origin. (a) A circle intersecting with a shifted plane. (b) Minimum distance between non-intersecting circle and plane.

SUBPROBLEM 4: CIRCLE AND PLANE

Given a vector p, unit vectors k, h, and a scalar d, find θ to minimize $\left|h^T \boldsymbol{R}(k,\theta)p - d\right|$.

Assume p and k are not collinear, and assume h and k are not collinear; otherwise, the problem does not depend on θ. If we apply (3), then

$$h^T \boldsymbol{R}(k,\theta)p - d = Ax - b \tag{14}$$

where $A = h^T A_{k,p}$ and $b = d - h^T k k^T p$. There are many solutions to $Ax = b$, which can be parameterized as $x = A^+ b + x_N$, where $A^+ = A^T(AA^T)^{-1}$ (for A with full row rank) is the pseudo-inverse (right inverse) of A, $x_{\min} = A^+ b$ is the minimum-norm solution, and x_N is an arbitrary vector in the null space of A, which is orthogonal to x_{min}. By assumption, the 2×1 row vector A is of rank one and has a one-dimensional null space given by

$$x_N = \xi\, x'_N, \quad x'_N = \underbrace{\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}}_{J} A^T, \tag{15}$$

where x'_N forms a basis for the null space and ξ parameterizes the solution. We can use $\|x\|^2 = 1$ to find ξ as the solution to a quadratic equation. The null space is the intersection between the plane containing the circle and a plane perpendicular to h. The minimum-norm solution x_{min} is given by

$$x_{min} = \frac{A^T}{\|A^T\|^2}\, b, \tag{16}$$

which is the point of intersection among three planes: The plane containing the circle, the plane perpendicular to dh, and the plane of symmetry spanned by k and h.

If $\|x_{min}\| < 1$, as in Fig. 4(a), two intersections occur, x_{min} corresponds to an interior point of the circle, and the two choices for ξ are

$$\xi = \pm\frac{\sqrt{1 - \|x_{min}\|^2}}{\|x'_N\|} = \pm\frac{\sqrt{\|A^T\|^2 - b^2}}{\|A^T\|^2}. \tag{17}$$

If $\|x_{min}\| = 1$, one intersection occurs, and $x = x_{min}$.

If $\|x_{min}\| > 1$, as in Fig. 4(b), the circle does not intersect with the plane. The minimizing solution occurs on the common normal which is perpendicular to the plane and passes through the circle axis. This means the solution occurs on the plane of symmetry, and x is therefore the normalized version of x_{min}. We can skip the normalization since we are plugging x into ATAN2.

If h is not a unit vector, the solution method stays the same. The only difference is in the geometric interpretation.

SUBPROBLEM 5: THREE CIRCLES

Given vectors, p_0, p_1, p_2, p_3 and unit vectors k_1, k_2, k_3, find $(\theta_1, \theta_2, \theta_3)$ to solve

$$p_0 + \boldsymbol{R}(k_1,\theta_1)p_1 = \boldsymbol{R}(k_2,\theta_2)(p_2 + \boldsymbol{R}(k_3,\theta_3)p_3). \tag{18}$$

We may visualize this subproblem as the intersection of three circles, as depicted in Fig. 5:

- Circle 1 is given by $p_0 + \boldsymbol{R}(k_1,\theta_1)p_1$.
- Circle 2 has axis k_2. Its radius r and height z are to be determined, where z may be negative if the circle center in the $-k_2$ direction.
- Circle 3 is given by $p_2 + \boldsymbol{R}(k_3,\theta_3)p_3$.

Assume (k_1, p_1) and (k_3, p_3) are not collinear, as otherwise θ_1 or θ_3 is arbitrary. (Other cases for a continuum of solutions exist if $p_0^T k_1^\times k_2 = 0$ or $p_2^T k_3^\times k_2 = 0$.) The projection of the solutions on circle 1 and circle 3 to k_2 is z:

$$z = k_2^T(p_0 + \boldsymbol{R}(k_1,\theta_1)p_1) = k_2^T(p_2 + \boldsymbol{R}(k_3,\theta_3)p_3). \tag{19}$$

Write

$$\begin{aligned} p_0 + \boldsymbol{R}(k_1,\theta_1)p_1 &= A_{k_1,p_1}x_1 + p_{1_S}, \\ p_2 + \boldsymbol{R}(k_3,\theta_3)p_3 &= A_{k_3,p_3}x_3 + p_{3_S}, \end{aligned} \tag{20}$$

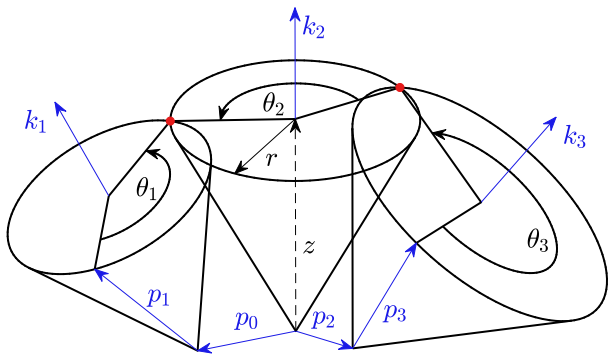


Fig. 5. Subproblem 5 finds the angles of intersections among three circles, where radius r and height z of circle 2 are to be determined and depend on θ_1 and θ_3. In the scenario depicted here, there exist other solutions of $(\theta_1, \theta_2, \theta_3)$ associated with different (r, z).

where the shifted centers of the circles are

$$p_{1_S} = p_0 + k_1 k_1^T p_1, \quad p_{3_S} = p_2 + k_3 k_3^T p_3. \tag{21}$$

Use Subproblem 4 to find x_1 and x_3 in terms of z:

$$x_i = \frac{v_i(z-\delta_i) \pm J v_i \sqrt{\|v_i\|^2 - (z-\delta_i)^2}}{\|v_i\|^2}, \tag{22}$$

where J is defined in (15), $v_i = A_{k_i,p_i}^T k_2$, and $\delta_i = k_2^T p_{i_S}$. By substituting (22) into (20) and taking the squared norm we obtain equations for two ellipses in $(r^2 + z^2, z)$, which are distorted versions of the cross sections of the surfaces of revolution of circles 1 and 3 around k_2:

$$\begin{aligned} r^2 + z^2 = &\left\|k_i^\times p_i\right\|^2 + \|p_{i_s}\|^2 + 2\alpha_i\delta_i - 2\alpha_i z \\ &\pm 2\beta_i \sqrt{\left\|k_i^\times p_i\right\|^2 \left\|k_i^\times k_2\right\|^2 - (z-\delta_i)^2}, \end{aligned} \tag{23}$$

where

$$\alpha_i = \frac{p_{i-1}^T {k_i^\times}^2 k_2}{\left\|k_i^\times k_2\right\|^2} \text{ and } \beta_i = \frac{p_{i-1}^T k_i^\times k_2}{\left\|k_i^\times k_2\right\|^2}. \tag{24}$$

To arrive at (23), we have used

$$A_{k_i,p_i} J A_{k_i,p_i}^T = \left\|k_i^\times p_i\right\|^2 k_i^\times, \tag{25}$$

$$\left\|A_{k_i,p_i}^T k_2\right\|^2 = \left\|k_i^\times p_i\right\|^2 \left\|k_i^\times k_2\right\|^2, \tag{26}$$

$$A_{k_i,p_i} A_{k_i,p_i}^T = -\left\|k_i^\times p_i\right\|^2 {k_i^\times}^2. \tag{27}$$

Equating the two right-hand sides of (23) and isolating and squaring for each radical yields a quartic polynomial in z. (In certain cases, it degenerates to a lower-degree polynomial.) There are many ways to find the roots of a quartic polynomial in closed form [46]. For each unique real root z_i, find solutions for x_1 and x_3 by computing the positive and negative branches in (22) and checking for equal norm after plugging into (20). Then, find θ_2 using Subproblem 1 to solve (18) using the value from (20). As two ellipses can intersect up to four times, there are up to four solutions for $(\theta_1, \theta_2, \theta_3)$.

Remark 6 (Coplanar Axes). The solution procedure simplifies when $p_0^T k_1^\times k_2 = 0$ or $p_2^T k_3^\times k_2 = 0$, meaning axis 1 or 3 is coplanar with axis 2. In this case, the equation for one of the ellipses degenerates into a line segment, and revolving circles 1 or 3 around k_2 results in a segment of a sphere or plane. If two axes intersect, rewrite the subproblem such that $p_0 = 0$ or $p_2 = 0$ and use Subproblem 3 and then Subproblem 2 to find up to four solutions. If two axes are parallel, use Subproblem 4 and then Subproblem 3 and Subproblem 1.

Remark 7 (Decomposition to Subproblem 6). As was done in [11], taking half the squared norm of both sides of (18) yields

$$\begin{aligned} &p_0^T \boldsymbol{R}(k_1,\theta_1)p_1 - p_2^T \boldsymbol{R}(k_3,\theta_3)p_3 \\ &\qquad = \frac{1}{2}(\|p_2\|^2 + \|p_3\|^2 - \|p_0\|^2 - \|p_1\|^2), \end{aligned} \tag{28}$$

and projecting (18) onto k_2 yields

$$k_2^T \boldsymbol{R}(k_1,\theta_1)p_1 - k_2^T \boldsymbol{R}(k_3,\theta_3)p_3 = k_2^T p_2 - k_2^T p_0. \tag{29}$$

These two equations may be solved with Subproblem 6. Then, θ_2 can be solved with Subproblem 1.

SUBPROBLEM 6: FOUR CIRCLES

Given vectors p_i, unit vectors k_i, h_i, $i = 1, \ldots, 4$, and scalars d_1, d_2, find (θ_1, θ_2) to solve

$$\begin{cases} h_1^T \boldsymbol{R}(k_1,\theta_1)p_1 + h_2^T \boldsymbol{R}(k_2,\theta_2)p_2 = d_1 \\ h_3^T \boldsymbol{R}(k_3,\theta_1)p_3 + h_4^T \boldsymbol{R}(k_4,\theta_2)p_4 = d_2. \end{cases} \tag{30}$$

This problem involves four circles defined by (k_i, p_i), where each circle also has a direction vector h_i. The position on the circle gets projected onto the h_i direction to get a distance for that circle. These circles are coupled so that the sums of distances for circles $(1, 2)$ and $(3, 4)$ must equal (d_1, d_2), and also coupled so that circles $(1, 3)$ and $(2, 4)$ have the same rotation angles (θ_1, θ_2).

We can use (3) to rewrite the subproblem as

$$\begin{bmatrix} h_1^T A_{k_1,p_1} & h_2^T A_{k_2,p_2} \\ h_3^T A_{k_3,p_3} & h_4^T A_{k_4,p_4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = Ax = b = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, \tag{31}$$

where

$$\begin{aligned} b_1 &= d_1 - h_1^T k_1 k_1^T p_1 - h_2^T k_2 k_2^T p_2, \\ b_2 &= d_2 - h_3^T k_3 k_3^T p_3 - h_4^T k_4 k_4^T p_4. \end{aligned} \tag{32}$$

If row 1 is a multiple of row 2, then A loses rank and there is a continuum of solutions. In this case, pick any nonzero equation from (30), choose θ_1 or θ_2 arbitrarily (within a valid range), and solve for the other joint angle with Subproblem 4.

If A is full rank, the unconstrained solutions to (31) are

$$x = x_{min} + \xi_1 x'_{N1} + \xi_2 x'_{N2}, \tag{33}$$

where $x_{min} = A^+ b$ is the minimum-norm solution, $x_N = \xi_1 x'_{N_1} + \xi_2 x'_{N_2}$ is is an arbitrary vector in the two-dimensional null space of A, x'_{N_i} forms a basis for the null space, and ξ_i parameterizes the solution. The values of x_{min}, x'_{N_1}, and x'_{N_2} can be found by QR decomposition of A^T.

If we impose the constraints $\|x_1\| = 1$ and $\|x_2\| = 1$, we get two equations for ellipses in (ξ_1, ξ_2). Like in Subproblem 5, the intersection of two ellipses can be found by finding the roots of a quartic polynomial. However, in this subproblem

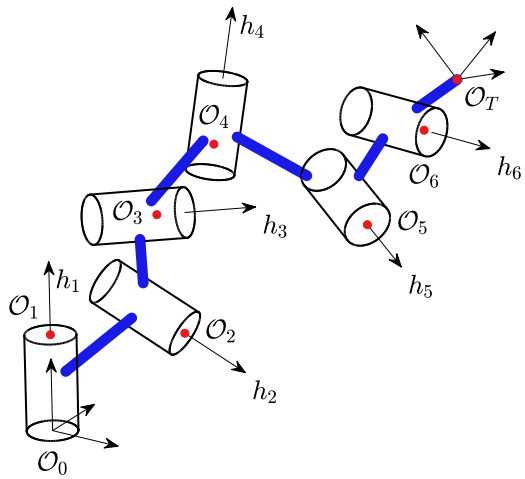


Fig. 6. General 6R robot arm with h_i as the ith joint axes and $\mathcal{O}_i$ as the origin of frame i, placed anywhere along h_i. Inverse kinematics can be solved using Subproblem 5 and a 2D search over two of the joint angles.

the equations for the ellipses are in standard form, so some extra work is needed to find the quartic equation [47]. The real solutions for (ξ_1, ξ_2) can then be used to find solutions to (x_1, x_2) and therefore the two unknown angles.

As in Subproblem 4, this solution method still works even if h_i are not unit vectors.

Remark 8 (Simplified Case). The problem simplifies if h_i or p_i is parallel to k_i for some i. In this case, one of the equations in (30) reduces to Subproblem 4. After solving for one angle from that equation, the other equation can be solved by using Subproblem 4 again.

V. Application to Robot Inverse Kinematics

Consider a six-dof articulated robot with all revolute joints as shown in Fig. 6. We will use the product of exponentials convention to describe the arm kinematics: Let frame 0 be the world coordinate frame, let frame $i, i = 1, \ldots, 6$ be the coordinate frame representing the orientation of link i which has equal orientation to frame 0 when the robot is in the zero configuration, and let frame T be the coordinate frame representing the orientation of the end effector tool. Let $\mathcal{O}_0$ be the origin of the world frame, let $\mathcal{O}_i$, $i = 1, \ldots, 6$, be the origin of frame i, placed anywhere along the ith joint axis, and let $\mathcal{O}_T$ be the origin of the end effector tool frame. Define $p_{i,j}$ ($p_{i,T}$), $i, j = 0, \ldots, 6$, be the $\mathbb{R}^3$ vector from $\mathcal{O}_i$ to $\mathcal{O}_j$ ($\mathcal{O}_T$) represented in the i frame. This means $p_{i-1,i}$, $i = 1, \ldots, 6$, and $p_{6,T}$ are constant $\mathbb{R}^3$ vectors.

Define h_i to be the constant unit vector rotation axis for the ith joint represented in the i frame, and let $R_{i-1,i} = \mathsf{R}(h_i, q_i)$, where q_i is the ith joint angle. Also let $R_{i,i-1} = \mathsf{R}(h_i, -q_i)$, and for $i < j$, $R_{ij} = R_{i,i+1} \cdots R_{j-1,j}$. Finally, define R_{6T} to be the constant orientation of the tool frame with respect to the 6 frame. Then, the rotation and position of the tool frame with respect to the world frame is

$$R_{0T} = R_{01}R_{12}R_{23}R_{34}R_{45}R_{56}R_{6T}, \tag{34a}$$

$$\begin{aligned} p_{0T} &= p_{01} + R_{01}p_{12} + R_{02}p_{23} + R_{03}p_{34} \\ &\quad + R_{04}p_{45} + R_{05}p_{56} + R_{06}p_{6T} \end{aligned} \tag{34b}$$

The inverse kinematics problem for a robot with kinematic parameters $\left(\{p_{i-1,i}\}_{i=1}^6,\ p_{6T},\ \{h_i\}_{i=0}^6,\ R_{6T}\right)$ means finding all joint angles $q = \begin{bmatrix} q_1 & q_2 & \cdots & q_6 \end{bmatrix}^T$ corresponding to a given end effector rotation and position (R_{0T}, p_{0T}). In general, one (R_{0T}, p_{0T}) corresponds to multiple q, which are called the different inverse kinematics branches.

Without loss of generality, assume $p_{01} = 0$. (Otherwise, subtract p_{01} from p_{0T}.) Rewriting (34) by moving constants to the left-hand side results in

$$R_{06} = R_{0T}R_{6T}^T = R_{01}R_{12}R_{23}R_{34}R_{45}R_{56}, \tag{35a}$$

$$\begin{aligned} p_{06} &= p_{0T} - R_{06}p_{6T} \\ &= R_{01}p_{12} + R_{02}p_{23} + R_{03}p_{34} \\ &\quad + R_{04}p_{45} + R_{05}p_{56}. \end{aligned} \tag{35b}$$

This means the IK procedures can be in terms of (R_{06}, p_{06}).

In the following inverse kinematics solutions using the subproblem decomposition method, we start with (35) and simplify based on any intersecting axes: if axes $i - i$ and i intersect, we can pick $\mathcal{O}_{i-1} = \mathcal{O}_i$, and so $p_{i-1,i} = 0$. Then, we decompose the equations into subproblems by moving terms and multiplying by rotations, taking the norm of both sides of the equation, or in the case of parallel axes projecting both sides of the equation onto a vector by left multiplying by h_i^T or right multiplying by h_i. When a subproblem has multiple solutions, the solution for q branches, meaning the procedures find all inverse kinematics solutions. (If desired, just one of the solutions can be used to speed up computation time.) If a subproblem has a continuum of solutions, such as when two rotation axes are collinear, then the robot has an internal singularity for that branch. This method is robust to such singularities as the arbitrary joint angles can be resolved before continuing the procedure. If a subproblem has no exact solutions, then that branch fails to find a solution; however, using the least-squares solution from Subproblems 1–4 may still be useful as described in section V-A3.

Each time we take the norm of both sides of an equation or project both sides of an equation onto a vector, we introduce the possibility for extraneous solutions. However, any extraneous solutions earlier in the branch become apparent in the next subproblems, as they do not return any exact solutions.

The following solutions are valid not just for a single robot, but for a family of robots with certain conditions on their kinematics parameters based on consecutive intersecting or parallel joints. Therefore, a single code implementation can perform inverse kinematics for multiple robots in the same kinematic family but with different dimensions.

In the solutions below we demonstrate specific locations for where interesting or parallel axes occur. However, similar equations can be easily derived for intersecting or parallel axes occurring elsewhere in the chain.

A. Closed-Form Inverse Kinematics

If a 6R robot has three consecutive intersecting or parallel joint axes, then the inverse kinematics can be solved using Subproblems 5 or 6, respectively. If a robot has certain other

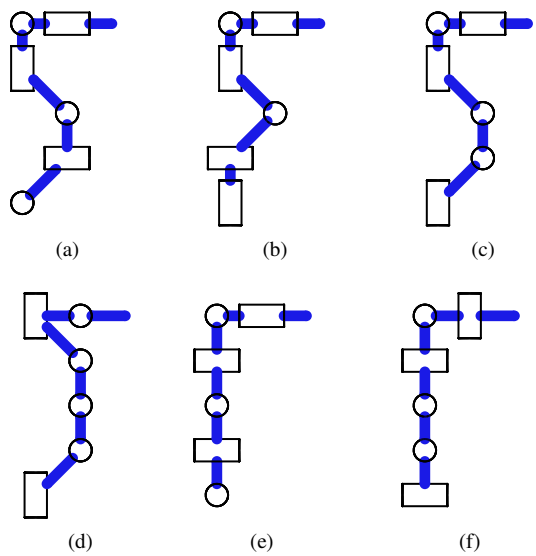


Fig. 7. Some cases for simplified robot inverse kinematics due to intersecting or parallel axes. (a) Spherical wrist. (b) Spherical wrist and two intersecting axes. (c) Spherical wrist and two parallel axes. (d) Three parallel axes. (e) Two intersecting axes. (f) Two parallel axes.

pairs of intersecting or parallel axes, then the solution can sometimes be written in terms of just Subproblems 1–4. This is in accordance with the special simplified cases of Subproblems 5 and 6.

1) Three Intersecting Joint Axes: When a 6R robot has three intersecting joint axes, the inverse kinematics problem can be decoupled into position and rotation parts. The position part can be solved with Subproblem 5, and the rotation part can be solved with Subproblems 1 and 2. Consider the common case of a spherical wrist where the last three joint axes (h_4, h_5, h_6) intersect at a point, shown in Fig. 7(a). Choose the origins of frames 4, 5, 6 at the intersection so $p_{45} = p_{56} = 0$. Then, (35b) becomes

$$-p_{12} + R_{10}p_{06} = R_{12}(p_{23} + R_{23}p_{34}), \tag{36}$$

which only depends on (q_1, q_2, q_3) and can be solved using Subproblem 5.

To solve for the wrist angles (q_4, q_5, q_6), write (35a) as

$$R_{32}R_{21}R_{10}R_{06} = R_{34}R_{45}R_{56} \tag{37}$$

For each (q_1, q_2, q_3), the left-hand side is known, which can be called R_{36}. Project it to h_6:

$$R_{43}R_{36}h_6 = R_{45}h_6, \tag{38}$$

and solve for up to two solutions of (q_4, q_5) using Subproblem 2. Then, find q_6 by using Subproblem 1 to solve (37). For a spherical wrist to achieve any orientation, it must have orthogonal consecutive joint axes.

If a robot has two intersecting axes separate from the three intersecting axes, then Subproblem 5 simplifies into Subproblems 2 and 3. Consider a robot with a spherical wrist and the first two axes intersecting as in Fig. 7(b). In this case, $p_{12} = 0$. Taking the norm of both sides of (36), we have

$$\|p_{06}\| = \|p_{23} + R_{23}p_{34}\|, \tag{39}$$

which may be solved using Subproblem 3 for up to two solutions of q_3. For each q_3, we can use Subproblem 2 to find up to two solutions of (q_1, q_2).

If a robot has two parallel axes separate from the three intersecting axes, then Subproblem 5 simplifies into Subproblems 1, 3 and 4. Consider a robot with a spherical wrist and the second and third axes parallel ($h_2 = h_3$) shown in Fig. 7(c). Rearranging (36) and projecting onto h_2, we have

$$h_2^T R_{10}p_{06} = h_2^T(p_{12} + p_{23} + p_{34}), \tag{40}$$

which may be solved using Subproblem 4 for up to two solutions of q_1. For each q_1, we can use Subproblem 3 to find up to two solutions of q_3. For each of the four possible (q_1, q_3), we can solve for the corresponding q_2 using Subproblem 1.

2) Three Parallel Joint Axes: When a 3R robot has three parallel axes, the inverse kinematics problem can be solved using Subproblems 1, 3, and 6. (Such a robot is the limit of a robot with three intersecting axes where the intersection point moves infinitely far away.) Consider a robot with $h_2 = h_3 = h_4$ as shown in Fig. 7(d). Projecting (35b) onto h_2 results in

$$h_2^T R_{10}p_{06} - h_2^T R_{45}p_{56} = h_2^T(p_{12} + p_{23} + p_{34} + p_{45}). \tag{41}$$

From (35a), we have

$$h_2^T R_{10}R_{06}h_6 - h_2^T R_{45}h_6 = 0 \tag{42}$$

If $p_{56} = 0$, we can find up to two solutions of q_1 from (41) using Subproblem 4. (A similar simplification occurs if a robot has three parallel axes and two other parallel axes.) For each solution of q_1, we can use Subproblem 4 to solve for up to two solutions of q_5 from (42) for a total of four solutions. If $p_{56} \neq 0$, (41) and (42) form a system of equations which can be solved using Subproblem 6 for up to four solutions of (q_1, q_5).

By projecting (35a) onto h_6 or h_2, we can solve for $q_2 + q_3 + q_4$ (and hence R_{14}) and q_6 using Subproblem 1 to solve

$$R_{14}R_{45}h_6 = R_{10}R_{06}h_6, \tag{43}$$

$$R_{65}R_{54}h_2 = R_{06}^T R_{01}h_2. \tag{44}$$

Find up to two solutions of q_3 using Subproblem 3 to solve

$$\|p_{23} + R_{23}p_{34}\| = \|R_{10}p_{06} - p_{12} - R_{14}p_{45} - R_{15}p_{56}\| \tag{45}$$

for a total of eight solutions. Within these eight solutions, we can solve for q_2 using Subproblem 1:

$$R_{12}(p_{23} + R_{23}p_{34}) = R_{10}p_{06} - p_{12} - R_{14}p_{45} - R_{15}p_{56}, \tag{46}$$

and we can solve for q_4 using subtraction, wrapping the result to $[-\pi, \pi]$ if desired.

3) Least-Squares Inverse Kinematics: We have seen above that in certain robots with sufficiently many intersecting or parallel axes, the IK problem can be solved using only Subproblems 1–4, and since these subproblems are posed as minimizations, the IK procedure can return solutions for q even for branches which may not have an exact solution. If a desired pose is not feasible but near the robot workspace, the IK procedure will return a close solution. This means the desired pose can be exactly on the boundary of the workspace

(i.e. at a boundary singularity), and the IK procedure will correctly find the robot joint angles even in the presence of numerical inaccuracies which may slightly perturb the desired pose outside workspace.

For certain robots with the task frame placed at the wrist, the least-squares solutions also solve the global least-squares problem in the usual Euclidean sense, solving the minimization

$$\min_q \left\| p_{0T}(q) - p_{0T}^{des} \right\| \text{ s.t. } R_{0T}(q) = R_{0T}^{des}, \tag{47}$$

where $(R_{0T}^{des}, p_{0T}^{des})$ is the desired end effector rotation and position. One robot configuration that achieves (47) is the common case of industrial robots similar to the ABB IRB 6640 [48]: a 6R robot with a spherical wrist ($p_{45} = p_{56} = 0$) and two parallel axes ($h_2 = h_3$), as well as $h_2^T(p_{12} + p_{23} + p_{34}) = 0$, $h_1^T h_2 = 0$, and $p_{6T} = 0$. We also need $h_4^T h_5 = h_5^T h_6 = 0$ so the spherical wrist can achieve any orientation. Under these conditions, we are minimizing distances in a cylindrical coordinate system. For some robots, such as when we have all the constraints above except $h_1^T h_2 \neq 0$, (47) is solved in only certain regions of the workspace.

B. Search-Based Inverse Kinematics

If at least one pair of consecutive joints are intersecting or parallel, then the problem can be reduced to a 1D search over a circle using Subproblems 5 or 6, respectively. If multiple pairs of consecutive joints are parallel or intersecting, then sometimes a 1D search may be performed using only Subproblems 1–4. In the fully general case where no consecutive joints are parallel or intersecting, then the problem reduces to a 2D search over a torus using Subproblem 5.

To perform a 2D search, we find the zeros of an error $e(q_i, q_j)$ which is the norm of a vector, meaning the zeros of this error are also local minima. Once the angles for joints i and j are found, the remaining four joint angles can be calculated in closed from. A straightforward way to find the zeros of the 2D error function is to sample the entire search space in a uniform grid and then perform numeric minimization starting with the smallest-valued sample. In MATLAB, the functions `fminsearch` or `lsqnonlin` can be used. Once a zero is found, any neighboring samples smaller than some threshold are removed from consideration, and another minimization can be performed starting at the next smallest-valued sample until all samples below some threshold are processed.

Performing a 1D search is even easier than performing a 2D search because the error $e(q_i)$ crosses zero. As before, we can sample the entire search space. Then, we find intervals where the error function switches sign: As long as the error function is defined and continuous throughout, zeros of the error function will be in this bracket. We can use a bracketing method such as the false position method, as summarized in [49], to quickly find the zero, then find the remaining joint angles in closed form. One edge case that this method does not account for is if the error function touches but does not cross zero.

For both 1D and 2D searches, the error function has multiple branches and may be undefined for some inputs, as shown in Fig. 8.

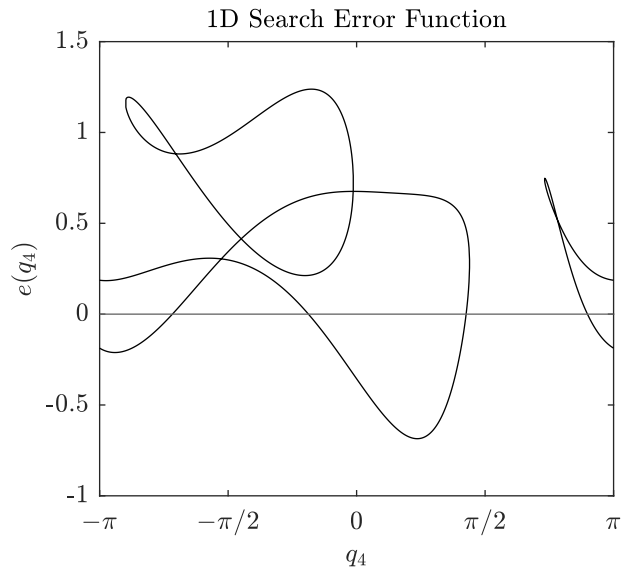


Fig. 8. Error function for a robot with axes 5 and 6 intersecting. For this end effector pose there are 4 IK solutions corresponding to the four zeros of the error function. There are four branches in this error function corresponding to the four solutions of Subproblem 5. For a given input, the error function has anywhere from zero values (e.g., $q_4 = \pi/2$) to four values (e.g., $q_4 = -\pi/2$).

If we only need to return a single solution for q and have an initial guess (e.g., to find a solution closest to a previous pose), finding the zero closest to the initial guess does not require sampling the whole search space. This dramatically speeds up the computation time. Computation time can also be reduced by only searching within joint limits. It may be useful to perform the search over joints with the smallest limits.

A caveat of these search-based methods is that certain solutions may only be found with a sufficiently small sampling interval. In a 2D search, a smaller sampling interval means finding zeros closer to the boundary of the domain. For a 1D search, a smaller sampling interval means finding zeros closer to points of zero or infinite slope. These points tend to correspond to singular configurations where small changes in one joint angle cause zero or infinite change to the end effector (and where different branches meet).

1) General Case (2D Search): IK for a general 6R robot with no intersecting or parallel axes can be solved by searching over any two joint angles. Given (q_1, q_2), we can solve for up to four solutions of (q_3, q_4, q_5) using Subproblem 5:

$$\begin{aligned} -p_{34} + R_{32}(R_{21}(R_{10}p_{06} - p_{12}) - p_{23}) \\ = R_{34}(p_{45} + R_{45}p_{56}). \end{aligned} \tag{48}$$

With each $(q_1, q_2, q_3, q_4, q_5)$, compute the error

$$e(q_1, q_2) = \left\| R_{01}R_{12}R_{23}R_{34}R_{45}h_6 - R_{06}h_6 \right\|. \tag{49}$$

Search through (q_1, q_2) to find the zeros of e. For each zero, use Subproblem 1 to solve

$$R_{56}p = R_{05}^T R_{06}p \tag{50}$$

for q_6, where p is any vector not collinear with h_6.

2) Two Intersecting Joint Axes (1D Search): A 6R robot with two intersecting axes can be solved by searching over one joint angle. If $p_{56} = 0$ as in Fig. 7(e), then (35b) becomes

$$-p_{12} + R_{10}p_{06} = R_{12}\left(p_{23} + R_{23}\left(p_{34} + R_{34}p_{45}\right)\right). \quad (51)$$

In this case, for each q_4, we can find up to four solutions of (q_1, q_2, q_3) using Subproblem 5. The error measures the solvability of

$$R_{45}R_{56} = R_{04}^T R_{06}. \quad (52)$$

By projecting onto h_5 and h_6, we get the error

$$e(q_4) = h_5^T R_{04}^T R_{06} h_6 - h_5^T h_6 \quad (53)$$

Perform a 1D search to find the zeros of e, and use Subproblem 1 to find q_5 and q_6 according to

$$R_{45}h_6 = R_{04}^T R_{06} h_6, \quad (54)$$
$$R_{65}h_5 = R_{06}^T R_{04} h_5. \quad (55)$$

3) Two Parallel Joint Axes (1D Search): Since a robot with two parallel axes is the limit of a robot with two intersecting axes where the point of intersection moves to a point infinitely far away, such a robot can also be solved with a 1D search. If $h_2 = h_3$ as in Fig. 7(f), then by substituting $R_{56} = R_{06}R_{65}$ and left multiplying by $h_2^T R_{10}$, (35b) becomes

$$\begin{aligned} h_2^T R_{10} R_{06} R_{65} p_{56} + h_2^T R_{34} p_{45} \\ = h_2^T (R_{10}p_{06} - p_{12} - p_{23} - p_{34}). \end{aligned} \quad (56)$$

Projecting (35a) onto h_2 and h_5 yields

$$h_2^T R_{10} R_{06} R_{65} h_5 - h_2^T R_{34} h_5 = 0. \quad (57)$$

Search over q_1 and use Subproblem 6 to find (q_4, q_6). Next, find $q_2 + q_3$ and therefore R_{13} with Subproblem 1 to solve

$$R_{13}R_{34}h_5 = R_{10}R_{06}R_{65}h_5. \quad (58)$$

The error measures the solvability of

$$\begin{aligned} R_{12}p_{23} = R_{10}p_{06} - p_{12} - R_{13}p_{34} \\ - R_{14}p_{45} - R_{10}R_{06}R_{65}p_{56} \end{aligned} \quad (59)$$

We search for q_1 to find the zeros of the error

$$\begin{aligned} e(q_1) = \|R_{10}p_{06} - p_{12} - R_{13}p_{34} \\ - R_{14}p_{45} - R_{10}R_{06}R_{65}p_{56}\| - \|p_{23}\|. \end{aligned} \quad (60)$$

Then solve for q_2 and q_5 with Subproblem 1, solving

$$h_2^T R_{34} R_{45} = h_2^T R_{10} R_{06} R_{65}, \quad (61)$$

$$R_{12}p_{23} = R_{10}p_{06} - p_{12} - R_{13}p_{34} - R_{14}p_{45} - R_{15}p_{56}, \quad (62)$$

and calculate q_3 with subtraction, wrapping the result to $[-\pi, \pi]$ if desired.

TABLE II
SUBPROBLEM SOLUTION RUNTIMES

Subproblem	Mean Runtime (ns)		
	m-file	MEX	Rust
1: Circle and Point	13 517	36	59
2: Two Circles	27 262	312	242
3: Circle and Sphere	18 034	134	111
4: Circle and Plane	17 669	105	115
5: Three Circles	105 288	2 553	1 027
6: Four Circles	116 750	3 110	1 436

TABLE III
HARDCODED ROBOT EXAMPLES

Robot Type	Hardcoded Example
General 6R	ABB YUMI [50], fixed q_3
Two intersecting axes	RRC K-1207i [51], fixed q_6
Two parallel axes	Two-Parallel-Bot
Spherical wrist	Spherical-Bot
and two intersecting axes	KUKA LBR iiwa 7 R800 [52], fixed q_3
and two parallel axes	ABB IRB 6640 [48]
Three parallel axes	Three-Parallel-Bot
and two intersecting axes	Universal Robots UR5 [53]

VI. EVALUATION

Suproblem and IK solutions have been implemented in MATLAB and Rust. Additionally, MATLAB code was compiled to C++ MEX code using MATLAB coder, which was then run from MATLAB.

The subproblem code will return all exact and least-squares solutions, and when applicable also return a flag to indicate if the answer is the least-squares solution. The five closed-form robot inverse kinematics solution programs will return all solutions and provide least-squares solutions when possible, along with flags to indicate if a solution is the least-squares type. The three search-based inverse kinematics programs will return multiple solutions. 1D search uses 200 initial samples, and 2D search uses a grid of 100 samples in each coordinate for a total of 10 000 samples. For the inverse kinematics solutions, computation time was also measured after hardcoding the robot kinematics parameters into the function, using one robot for each of the inverse kinematics algorithms. The eight robot examples (Table III) include real 6-dof robots as well as real 7-dof robots with one fixed joint angle, and three robots are fictitious (Spherical-Bot, Two-Parallel-Bot and Three-Parallel-Bot).

Timing comparisons were performed using the following procedure for all implementations: For each subproblem or IK problem, a file with 10 000 randomly generated test cases (or fewer for certain IK tests), each with at least one exact solution, is loaded into memory and the program is run. The execution time and the returned solutions for all test cases were saved.

Timing comparisons were also made to two other publicly available analytical IK solvers: IKFast and the MATLAB Robotics Toolbox solver. IKFast generated valid code for three types of robots but failed in the other five cases. Generating and compiling code for the UR5 took an exceptionally long time: 55 min and 10 min, respectively. The MATLAB solver

TABLE IV
ROBOT INVERSE KINEMATICS TIMING

Robot Type	Mean Random Robot Runtime (µs)			Mean Hardcoded Robot Runtime (µs)			
	m-file	MEX	Rust	MEX	Rust	IKFast	MRT MEX
General 6R (2D search)	1 490 518.240*	27 904.571**	18 808.249	20 741.535**	16 026.010	N/A[a]	N/A[a]
Two intersecting axes (1D search)	35 592.452**	659.130	339.214	199.194	114.552	N/A[a]	N/A[a]
Two parallel axes (1D search)	58 483.194**	954.959	476.725	1 021.655	439.445	N/A[a]	N/A[a]
Spherical wrist	489.893	5.158	2.296	4.527	2.425	111.445	10.583
and two intersecting axes	587.320	3.423	2.062	4.991	3.688	N/A[b]	N/A[a]
and two parallel axes	745.547	3.349	2.193	2.991	3.183	4.417	13.318
Three parallel axes	391.796	5.762	3.198	5.345	4.339	N/A[c]	N/A[a]
and two intersecting axes	376.290	2.981	2.232	2.846	3.089	121.714	N/A[a]

* 100 trials ** 1000 trials [a] Did not generate code [b] Did not compile [c] Incorrect IK

could only find solutions for two robots. Although the KUKA R800 (with fixed q_3) is a robot with spherical wrist, the `generateIKFunction` function crashed.

Timing results are shown in Tables II and IV. Timing trials were performed with MATLAB R2023a and `rustc` 1.70.0. The measurement was performed on a computer with an Intel Core i7-3770K CPU at 3.50 GHz and 16 GB memory running Windows 10. Rust timing was done using Windows Subsystem for Linux with Ubuntu 18.04.

Subproblem solutions using MEX and Rust ran in under 4000 ns, with Subproblems 1–4 faster than Subproblems 5 and 6. m-file implementations were about 2 orders of magnitude slower. Closed-form IK solutions using the subproblem decomposition method ran in under 6 µs for all MEX and Rust implementation, with many solutions running in under 3 µs. Although m-file implementations ran slower by about two orders of magnitude, they are still fast enough for real-time control. Search-based IK is slower than closed-from IK by a factor roughly equal to the number of initial search samples: 200 for 1D search and 10 000 for 2D search. Hardcoding the kinematics often sped up computation time for MEX code. This is most apparent for the RRC arm since in this case Subproblem 5 is solved using a quadratic rather than a quartic. Unexpectedly, Rust timing was usually slower for hardcoded kinematics. This is worth investigating as having constants and multiplications by zero should lead to faster compiled code. MEX and Rust implementations of subproblem decomposition are faster and can solve more types of robots than IKFast and Matlab Robotics Toolbox. For example, IK for a UR5 robot using subproblem decomposition is more than 40 times faster than IKFast.

VII. CONCLUSION

This paper revisits the subproblem decomposition method for solving robot inverse kinematics. We found closed-form solutions to the original three Paden–Kahan subproblems as well as three additional subproblems, and for Subproblems 1–4, we found the least-squares solutions in closed form when the exact problem has no solution. Our approach converts a nonlinear subproblem to a linear one and then imposes constraints on the linear solutions. It is easy to understand and visualize and is computationally efficient. The inverse kinematics for any robot with three consecutive intersecting or parallel axes may be decomposed into a series of subproblems and solved directly in closed form. For other robots, the decomposition method reduces the solution to a 1D or 2D search. This method finds *all* inverse kinematics solution instead of just one solution close to an initial guess as in Jacobian-based methods.

We have demonstrated good computational performance for IK using the subproblem decomposition method, and there are still ways to speed up computation time. The implementation code for most IK solutions can be better optimized as most of computation time is spent not on solving subproblems but on other calculations such as computing rotation matrices. Perhaps performance can be improved by computing rotations using quaternions or by directly using s_θ and c_θ from each subproblem solution. For search-based IK solutions, there are a number of possible algorithmic improvements. Sampling could be parallelized, or a much faster search method could be used, such as one that immediately starts optimization after just a few random samples, doing a random restart if that optimization fails. It may also improve performance to increase sampling in regions next to the boundary of the domain or in regions that may be local minima or maxima pointing towards zero. For 1D searches, in principle it should be possible to use the tangent half angle substitution to convert e into a polynomial, allowing for use of polynomial root finding algorithms. For robots which cannot be solved in closed form using the subproblem decomposition method, rather than performing a 1D or 2D search, it may be useful to use existing general 6-dof methods to find one or two joint angles, and then use subproblem decomposition to quickly and easily find all remaining joint angles in closed form.

There are still certain robots with closed-form solutions which cannot be solved using these subproblems. For example, a 2R-2R-2R robot with perpendicular axes at each 2R joint does have a closed-form solution [4] but cannot be solved with these subproblems at this point.

There are applications for these subproblems beyond just inverse kinematics for 6R arms. We can use these subproblems to solve inverse kinematics for 7-DOF arms if some parameterization for the redundant degree of freedom is also given (e.g., elbow angle). These subproblems may also be useful for other geometric problems, such as forward kinematics for parallel manipulators, the elbow angle for 7-dof arms, or

certain problems found in computer graphics and animation.

VIII. Acknowledgments

The authors would like to thank C. Carignan, D. Carabis, and A. Gostin for their fruitful discussions. The authors would also like to thank R. Chen, A. Maksumič, J. Mathew, and A. Ropp for their careful work in implementing the subproblem and IK solutions. Finally, the authors would like to thank the reviewers of this paper for their comprehensive comments and recommendations.

References

[1] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2015, pp. 928–935.

[2] R. Diankov, "Automated construction of robotic manipulation programs," Ph.D. dissertation, Carnegie Mellon University, Robotics Institute, August 2010. [Online]. Available: http://www.programmingvision.com/rosen_diankov_thesis.pdf

[3] I. A. Sucan and S. Chitta, "MoveIt," moveit.ros.org, (accessed Jun. 14, 2023).

[4] D. L. Pieper, *The kinematics of manipulators under computer control*. Stanford University, 1969.

[5] M. Raghavan and B. Roth, "Kinematic analysis of the 6R manipulator of general geometry," in *International symposium on robotics research*. Citeseer, 1990, pp. 314–320.

[6] ——, "Inverse kinematics of the general 6R manipulator and related linkages," 1993.

[7] H. Lee, C. Woernle, and M. Hiller, "A complete solution for the inverse kinematic problem of the general 6R robot manipulator," 1991.

[8] D. Kohli and M. Osvatic, "Inverse kinematics of general 6R and 5R, P serial manipulators," 1993.

[9] D. Manocha and J. F. Canny, "Efficient inverse kinematics for general 6R manipulators," *IEEE transactions on robotics and automation*, vol. 10, no. 5, pp. 648–657, 1994.

[10] M. L. Husty, M. Pfurner, and H.-P. Schröcker, "A new and efficient algorithm for the inverse kinematics of a general serial 6R manipulator," *Mechanism and machine theory*, vol. 42, no. 1, pp. 66–81, 2007.

[11] J. Angeles, *Fundamentals of robotic mechanical systems: theory, methods, and algorithms*. New York: Springer, 2014.

[12] C. Mavroidis and B. Roth, "Structural parameters which reduce the number of manipulator configurations," 1994.

[13] B. E. Paden, "Kinematics and control of robot manipulators," Ph.D. dissertation, EECS Department, University of California, Berkeley, Jan 1986. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/1986/631.html

[14] R. Murray, Z. Li, and S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. Boca Raton, FL: CRC Press, 1994.

[15] H. Wang, X. Lu, C. Sheng, Z. Zhang, W. Cui, and Y. Li, "General frame for arbitrary 3R subproblems based on the POE model," *Robotics and Autonomous Systems*, vol. 105, pp. 138–145, 2018.

[16] T. Song, B. Pan, G. Niu, J. Yan, and Y. Fu, "General closed-form inverse kinematics for arbitrary three-joint subproblems based on the product of exponential model," *Frontiers of Mechanical Engineering*, vol. 17, no. 2, pp. 1–17, 2022.

[17] P. Zsombor-Murray and A. Gfrerrer, "3r wrist positioning-a classical problem and its geometric background," in *Proc. of the 5th Int. Workshop on Computational Kinematics, Springer, Berlin*, 2009, pp. 175–182.

[18] I. Dimovski, M. Trompeska, S. Samak, V. Dukovski, and D. Cvetkoska, "Algorithmic approach to geometric solution of generalized Paden–Kahan subproblem and its extension," *International Journal of Advanced Robotic Systems*, vol. 15, no. 1, 2018.

[19] T. Yue-sheng and X. Ai-ping, "Extension of the second Paden–Kahan sub-problem and its' application in the inverse kinematics of a manipulator," in *2008 IEEE Conference on Robotics, Automation and Mechatronics*, 2008, pp. 379–381.

[20] Q. Chen, S. Zhu, and X. Zhang, "Improved inverse kinematics algorithm using screw theory for a six-DOF robot manipulator," *International Journal of Advanced Robotic Systems*, vol. 12, no. 10, p. 140, 2015.

[21] H. S. An, T. W. Seo, and J. W. Lee, "Generalized solution for a subproblem of inverse kinematics based on product of exponential formula," *Journal of Mechanical Science and Technology*, vol. 32, no. 5, pp. 2299–2307, 2018.

[22] H. Wang, X. Lu, Z. Zhang, Y. Li, C. Sheng, and L. Gao, "A novel second subproblem for two arbitrary axes of robots," *International Journal of Advanced Robotic Systems*, vol. 15, no. 2, 2018.

[23] M. Kong, Z. Du, L. Sun, and Y. Zhang, "Solution and application of two inverse kinematics subproblems," in *2006 International Conference on Mechatronics and Automation*. IEEE, 2006, pp. 1164–1168.

[24] J. Xu, Z. Liu, Q. Cheng, Y. Zhao, Y. Pei, and C. Yang, "Models for three new screw-based IK sub-problems using geometric descriptions and their applications," *Applied Mathematical Modelling*, vol. 67, pp. 399–412, 2019.

[25] I.-M. Chen and Y. Gao, "Closed-form inverse kinematics solver for reconfigurable robots," in *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164)*, vol. 3. IEEE, 2001, pp. 2395–2400.

[26] J. Leoro, T. Hsiao, and C. Betancourt, "A new geometric subproblem to extend solvability of inverse kinematics based on screw theory for 6R robot manipulators," *International Journal of Control, Automation and Systems*, vol. 19, no. 1, pp. 562–573, 2021.

[27] J. Zhao, W. Wang, Y. Gao, and H. Cai, "Generation of closed-form inverse kinematics for reconfigurable robots," *Frontiers of Mechanical Engineering in China*, vol. 3, no. 1, pp. 91–96, 2008.

[28] H. Wang, X. Lu, W. Cui, Z. Zhang, Y. Li, and C. Sheng, "General inverse solution of six-degrees-of-freedom serial robots based on the product of exponentials model," *Assembly Automation*, 2018.

[29] E. Sariyildiz and H. Temeltas, "A comparison study of three screw theory based kinematic solution methods for the industrial robot manipulators," in *2011 IEEE International Conference on Mechatronics and Automation*. IEEE, 2011, pp. 52–57.

[30] Y. S. Tan, P. L. Cheng, and A. P. Xiao, "Solution for a new subproblem in screw theory and its' application in the inverse kinematics of a manipulator," in *Applied Mechanics and Materials*, vol. 34. Trans Tech Publ, 2010, pp. 271–275.

[31] ——, "Inverse kinematics solution for a 6R special configuration manipulators based on screw theory," in *Advanced Materials Research*, vol. 216. Trans Tech Publ, 2011, pp. 250–253.

[32] J. M. Pardos-Gotor, *Screw Theory in Robotics: An Illustrated and Practicable Introduction to Modern Mechanics*. CRC Press, 2021.

[33] P.-F. Lin, M.-B. Huang, and H.-P. Huang, "Analytical solution for inverse kinematics using dual quaternions," *IEEE Access*, vol. 7, pp. 166 190–166 202, 2019.

[34] L. Chen, T. Zielinska, J. Wang, and W. Ge, "Solution of an inverse kinematics problem using dual quaternions," *International Journal of Applied Mathematics and Computer Science*, vol. 30, no. 2, 2020.

[35] L. Josuet, B. Carlos, L. Hsien, H. Te-Sheng, W. Chun-Sheng *et al.*, "An improved inverse kinematics solution of 6R-DOF robot manipulators with euclidean wrist using dual quaternions," in *2016 International Automatic Control Conference (CACS)*. IEEE, 2016, pp. 77–82.

[36] M. D. Shuster and F. L. Markley, "Generalization of the Euler angles," *The Journal of the Astronautical Sciences*, vol. 51, no. 2, pp. 123–132, 2003.

[37] W. I. Clement, H. H. An, and B. Reed, "Analytical shoulder-elbow-wrist inverse kinematics for the 7-dof Motoman SIA50D robot arm," NASA, Tech. Rep., 2013.

[38] M. Pfurner, "Explicit algebraic solution of geometrically simple serial manipulators," in *Computational Kinematics: Proceedings of the 5th International Workshop on Computational Kinematics*. Springer, 2009, pp. 167–174.

[39] "Tesseract," https://github.com/tesseract-robotics/tesseract, (accessed Jun. 14, 2023).

[40] MATLAB, "Robotics system toolbox," https://www.mathworks.com/products/robotics.html, (accessed Jun. 14, 2023).

[41] A. Norrdine, "An algebraic solution to the multilateration problem," in *Proceedings of the 15th International Conference on Indoor Positioning and Indoor Navigation, Sydney, Australia*, vol. 1315, 2012.

[42] L. Euler, "Nova methodus motum corporum rigidorum degerminandi," *Novi commentarii academiae scientiarum Petropolitanae*, pp. 208–238, 1776.

[43] O. Rodrigues, "Des lois géométriques qui régissent les déplacements d'un système solide dans l'espace, et de la variation des coordonnées provenant de ces déplacements considérés indépendamment des causes qui peuvent les produire," *J. Math. Pures Appl*, vol. 5, no. 380-400, p. 5, 1840.

[44] P. Zsombor-Murray, M. Hayes, and M. Husty, "Extreme distance to a spatial circle," *Transactions of the Canadian Society for Mechanical Engineering*, vol. 28, no. 2A, pp. 221–235, 2004.

[45] B. Odehnal, "Common normals of two tori," *Journal for Geometry and Graphics*, vol. 9, no. 1, pp. 51–65, 2005.

[46] S. L. Shmakov, "A universal method of solving quartic equations," *International Journal of Pure and Applied Mathematics*, vol. 71, no. 2, pp. 251–259, 2011.

[47] D. Eberly, "Intersection of ellipses," Geometric Tools, Redmond WA, Tech. Rep., September 2020, Accessed May 23, 2023. [Online]. Available: https://www.geometrictools.com/Documentation/IntersectionOfEllipses.pdf

[48] ABB, "IRB 6640," https://new.abb.com/products/3HAC020536-012/irb-6640, 2022, (accessed Oct. 31, 2022).

[49] S. Abbasbandy and S.-j. Liao, "A new modification of false position method based on homotopy analysis method," *Applied Mathematics and Mechanics*, vol. 29, pp. 223–228, 2008.

[50] ABB, "IRB 14000 YuMi - collaborative robot," https://new.abb.com/products/robotics/collaborative-robots/yumi/irb-14000-yumi, 2022, (accessed Oct. 31, 2022).

[51] Robotics Research Corporation, "Dexterous manipulators and advanced control systems," Robotics Research Corporation, Tech. Rep., 2005, Accessed Oct. 31, 2022. [Online]. Available: http://www.robotics-research.com/RRCTechDoc.PDF

[52] KUKA, "LBR iiwa," https://www.kuka.com/en-us/products/robotics-systems/industrial-robots/lbr-iiwa, 2022, (accessed Oct. 31, 2022).

[53] Universal Robots, "UR5 collaborative robot arm," https://www.universal-robots.com/products/ur5-robot/, 2022, (accessed Oct. 31, 2022).